

conditions verwenden, Funktionen

| Vergleich zweier signed Zahlen |                          |                                |    |
|--------------------------------|--------------------------|--------------------------------|----|
| Fall                           | Flags                    | Begründung                     | CC |
| $a = b$                        | $ZF = 1$                 | $a - b = 0$                    | E  |
| $a < b$                        | $OF \neq SF$             | $SF = 0 \vee 1$ oder umgekehrt | L  |
| $a \geq b$                     | $OF = SF$                | $a \geq b$ heisst $a < b$      | GE |
| $a \leq b$                     | $ZF = 1 \vee OF \neq SF$ | $\leq$ heisst $<$ oder $=$     | LE |
| $a > b$                        | $ZF = 0 \vee OF = SF$    | $a > b$ heisst $a \leq b$      | G  |

Beispiel:  $-10 - (+5) = -15 \Rightarrow SF = 1, OF = 0, +5 - (-10) = -5 \Rightarrow OF = 1, SF = 1$

-----

## Bedingte Anweisungen

**cmp:**  
Für Vergleiche kann die Operation **cmp** eingesetzt werden. Funktioniert gleich wie **sub**, ausser dass die verglichenen Operanden nicht verändert werden.

```
mov rax, [ux]; mov rbx, [uy]; cmp rax, rbx;  
cmovle rcx, 5 ; conditional move equal, only if rax == rbx
```

```
test: (wie cmp, aber mit Overflow flag und AND & statt Subtraktion)
test rax, rax
cmovz rax, rbx ; conditional move zero, only if result == 0
```

3 Familien von bedingten Operationen bei Intel:

- CMOVcc**: Conditional MOV
- Jcc**: Conditional JMP (Jump)

**SETcc:** Schreibt 1 in das 8-Bit grosse Ziel, wenn CC (*Bedingung*) erfüllt, sonst 0

Der Befehl `JMP d` **springt** im Programmfluss um `d`, **Offset ist relativ** zum Index der Instruktion, die **eigentlich ausgeführt**

werden wurde. Um nicht mit dem Index arbeiten zu müssen, können **Labels** verwendet werden: `jmp target` springt bis zum Label `target`. Dies ist **flexibler** und **stabiler** als die

Verwendung eines Offsets.

**Bedingte Sprünge**

33A `mov rax, rbx`

Benötigen einen Condition Code. Springen nur dann, wenn der CC erfüllt ist. Ansonsten fährt der Prozessor mit der nächsten Instruktion fort.

```
mov eax, [v]
```

```
cmp eax, 3 ; is eax == 3?
jne if_not_3 ; jump if not equal
```

```
mov eax, [y] ; wird ausgeführt, wenn eax == 3
if_not_3:
    inc eax ; wird ausgeführt, wenn eax != 3
```

Wird in C mit **if** implementiert. Der Body **wird ausgeführt**, wenn die **Bedingung nicht 0** ist.

Der Compiler verwendet direkt die Condition Codes, wenn möglich.

**Verzweigung mit Alternative (if ... else)**

Das Flag `Redu` wird gesetzt, wenn ausgeführt, wenn die Bedingung 0 ist.

Die Reihenfolge der Bodies kann im Assembler vertauscht sein.  
Am Ende des ersten Bodies kommt ein unbedingter Sprung.

```
if (ux < 3) { /* if-body */ } else { /* else-body */ }
```

wird zu:

```
mov eax, [ux]; cmp eax, 2
ja else_body ; jump if above 2
; if-body
```

```
    jmp after_if
else_body: ; else-body
after_if:
```

**Schleifen (Loops)**  
Können mit Rückwärtssprung implementiert werden.

**Do-While-Schleife:** Nach jedem Ausführen des Bodys wird die Bedingung überprüft. Wird mindestens einmal ausgeführt.

|                                            |                                         |
|--------------------------------------------|-----------------------------------------|
| <pre>int i = 23; do {     /* body */</pre> | <pre>mov rcx, 23 loop:     ; body</pre> |
|--------------------------------------------|-----------------------------------------|

```
} while (--i);      | dec rcx; jnz loop ; jump if not zero
```

**While-Schleife:** Vor jedem Ausführen des Bodys wird die Bedingung überprüft

```
int i = 23;      mov rcx, 23
```

```
// == while (--i != 0)      jmp condition ; check condition
while (--i) {              loop:
    /* body */              ; body
```

```
condition:
    dec rcx; jnz loop ; jump if not zero
```

### Pointer-Addition

Die Addition eines Integers zu einem `T*`-Pointer **berücksichtigt** die Grösse `sizeof(T)`

**Beispiel:** sizeof(int) = 4 bytes. Bei int \*p = 0x20; ist p+1 == 0x24 und p+2 == 0x28

**Beispiel Summieren über Speicherbereich**

```
extern int *begin; // points to first element
extern int *end;   // points *after* last element
int sum = 0;       // total sum of all elements
```

```
int sum = 0; // total sum of all elements
for (
    int *p = begin; // init with address of first element
```

```

    p != end;      // execute loop while not after end
    ++p           // set to address of next element
} if sum += *p; } // add content of p

```

```

    )(sum += xp, ) // add content of p

```

## 5. FUNKTIONEN

Adressen stehen nicht in den Instruktionen, sondern in **Register** oder globalen **Variablen**.  
(Variablen sind etwas langsamer), jedoch kein Problem mehr mit zu wenig Register.

**Jedoch:** Für jede Sequenz muss Speicher statisch reserviert werden, für jede Sequenz gibt es jeden Parameter genau einmal. Keine Rekursion (gibt Endlosschleife) und keine Parallelität.

### Stack

Auf dem Stack können Zwischenwerte, Argumente, Rückgabewerte, Rücksprungadresse gespeichert werden.

**Stack wächst nach unten:** Stack wächst in Richtung der niedrigeren Adressen, Das «oberste»

Stack Register  
Register Basepointer (rbp) Frame Pointer: Beginn der Stacks (an höchster Adresse)

**Register Stackpointer (rsp):** Aktuelle Adresse im Stack.  
Funktionsaufruf: der rbp wird auf den Stack gepusht (push rbp) und der aktuelle rsp zum rbp

**"mov rbp, rsp" (ProLog).** So können Adressen relativ zum **rbp** angegeben werden. Beim Verlassen der Funktion wird das Ganze wieder rückgängig gemacht (Epilog) **"mov rsp, rbp; pop rbp"**.

**push** und **pop** (müssen immer gleichmässig verwendet werden.)  
**push q** kopiert das Element aus q oben auf den Stack und dekrementiert Stackpointer.  
**pop z** kopiert das oberste Element vom Stack nach z und inkrementiert Stackpointer (Wert auf

**push rax** entspricht: | **pop rax** entspricht:

|                               |                              |
|-------------------------------|------------------------------|
| sub rsp, 8 ;decrement Pointer | mov rax, [rsp] ;get Value    |
| mov [rsp], rax ;insert Value  | add rsp, 8;increment Pointer |

